38-1-S Hoďte si kostkou

Úkol 1:

Pro simulaci l-stěnné kostky podle zadání nejdříve budeme potřebovat implementovat k-stěnnou spolu s poctivou mincí. To uděláme podle popisu úlohy velice jednoduše. Vzhledem k faktu, že v zadání už jsou tyto funkce popsané, tak pouze u nich ukážu kód, který je velice triviální a odpovídá chování, které je organizátory popsáno. Všechen ukázkový kód bude v C++.

Poctivá mince:

```
template<typename T>
T RandomProbability(T min, T max) {
   random_device rd;
   mt19937 gen(rd());
   uniform_real_distribution<> dis(min, max);
   return dis(gen);
}
int PerfectCoinFlip() {
   double flip = RandomProbability<double>(0, 1);
   if (flip >= 0.5) {
       return 1;
   }
   return 0;
}
```

Vygenerujeme náhodné $x \in [0, 1)$ a podle hodnoty vrátíme buď 1 nebo 0.

K-stěnná kostka:

```
int CubeKSimulation(int k) {
   if (k < 2) { cout << "K < 2!!!" << endl; return 0; }
   double b = log2(k);
   double b_2 = (floor(b) != b) ? floor(b) + 1 : b;
   int val = k;

   while (val >= k) {
       string str_bin;
       for (int i = 0; i < b_2; i++)
           str_bin += to_string(PerfectCoinFlip());
       val = stoi(str_bin, nullptr, 2);
   }
   return val;
}
```

Odpovídá popisu v zadání a splňuje poctivou pravděpodobnost $P = \frac{1}{k}$ a asymptotickou složitost $O(log_2 k)$

L-stěnná kostka:

Pro simulaci l-stěnné kostky můžeme použít úplně stejný přístup jako pro simulaci k-stěnné kostky poctivou mincí. Ovšem namísto generování bitů mincí použijeme naší původní kostku. Mocnina dvojky *b* pro kterou musí platit $l \leq 2^b$ bude reprezentovat počet platných hodů kostkou, ze kterých vygenerujeme naše bity. Tyto bity si budeme průběžně ukládat do řetězce a na konci je převedeme do desítkové soustavy a vrátíme v případě, kdy platí, že číslo je menší než hodnota l. Pokud podmínka neplatí, všechny bity zahodíme a generujeme znovu od začátku.

Jestli hod kostkou vygeneruje bit hodnoty 0 nebo 1, bude určovat hodnota i-tého hodu k-stěnné kostky. A to konkrétně tak, že v situaci, kdy jak pro sudý, tak i lichý počet stěn *k* padne hodnota menší než $\frac{k}{2}$, bude odpovídat bitu 0. Bitům 1 naopak budou odpovídat hody jejichž hodnoty jsou větší než $\frac{k}{2}$ pro lichý počet stěn *k* a větší nebo rovny pro sudý počet stěn *k*. Střední hodnotu u lichého počtu stěn zahodíme a jednoduše hodíme znovu, abychom zajistili stejné pravděpodobnosti a tedy, že l-stěnná kostka bude poctivá.

```
int CubeLSimulationUsingCubeKBinnaryApproach(int l, int k) {
   if (l < 2 || k < 2) {
      cout << "K < 2 nebo l < 2!!!" << endl;
      return 0;
   }
   double b = log2(l);
   double b_2 = (floor(b) != b) ? floor(b) + 1 : b;
   int val = l;

   while (val >= l) {
       string str_bin;
       for (int i = 0; i < b_2; i++) {
           int roll = CubeKSimulation(k);
           if (roll < k/2) str_bin += "0";
           else if ((roll >= k/2 && k % 2 == 0) ||
                    (roll > k/2 && k % 2 != 0)) str_bin += "1";
           else i--;
       }
       val = stoi(str_bin, nullptr, 2);
   }
   return val;
}
```

Důkazem správnosti pak budiž fakt, že vše vychází z rovnoměrné pravděpodobnosti poctivé mince. Stejně jako u k-stěnné kostky, kdy v případě překročení nebo dosažení hodnoty *k* vše zahodíme a začneme s generováním bitů, které mají rovnoměrnou pravděpodobnost, od začátku, se tomu děje stejně i u l-stěnné. Všimněte si, že rovnoměrné pravděpodobnosti při generování bitů k-stěnnou kostkou dosáhneme stejně jako u poctivé mince, respektive první polovina hodnot, které mohou padnout odpovídá bitu 0 a druhá bitu 1. Platí tomu i v případě, kdy má k-stěnná kostka lichý počet stěn, protože pokud střední hodnota pro liché *k* u i-tého hodu padne, tak abychom zabránili porušení rovnoměrné distribuci pravděpodobnosti, tento hod zcela vynecháme a házíme znovu. Proto aby byla porušena rovnoměrnost určování bitů k-stěnnou kostkou a tedy i poctivost l-stěnné kostky, musela by být porušena samotná rovnoměrnost pro k-stěnnou kostku, tedy pro poctivou minci. To ale ovšem není pravda, jelikož máme dokázané, že poctivá mince je poctivá a proto je zaručené, že taktéž i l-stěnná kostka bude skutečně poctivá.

Abychom spočítali pravděpodobnost úspěšného pokusu, budeme vycházet z toho, co už víme o naší k-stěnné kostce. Tedy, že pravděpodobnost úspěšného pokusu pro k-stěnnou kostku je $p = \frac{k}{2^b}$. Naší k-stěnnou kostku využíváme pro generování bitů stejným způsobem jako minci, jen s tím rozdílem, že v případě kdy bude lichý počet stěn *k* a padne prostřední hodnota, hod zcela vynecháme a házíme znovu. Proto pravděpodobnost úspěchu bude pro l-stěnnou kostku skoro stejná $p = \frac{l}{2^b}$.

Pokud chceme zjistit střední hodnotu, tedy kolikrát kostkou bude třeba hodit. Lemma o džbánu i zadání říká, že kvůli nezávislosti jevů bude střední hodnota (počet hodů) pro úspěšný pokus $\frac{1}{p}$. Pro lichý počet *k* ovšem pravděpodobnost úspěchu bude nižší, konkrétně $\frac{k-1}{k}$ jelikož střed vynecháme (označme P_s a P_l → pravděpodobnost sudého/lichého *k*). Označme si proměnnou C pro počet hodů k-stěnnou kostkou a U pravděpodobnost úspěšného hodu.

$$E[C] = P_{s/l} \cdot b \cdot E[U] = P_{s/l} \cdot b \cdot \frac{1}{p} = P_{s/l} \cdot b \cdot \frac{1}{\frac{l}{2^b}} = P_{s/l} \cdot \frac{2^b}{l} \cdot b$$

$$Pro\ P_s = 1; E[U] = \frac{2^b}{l} \cdot b \cdot 1;\ 2^{b-1} < l < 2^b \Rightarrow \frac{1}{2} < \frac{l}{2^b} < 1; b \cdot 1 < \frac{2^b}{l} \cdot b \cdot 1 < 2b \cdot 1$$

$$Pro\ P_l = \frac{k-1}{k}; E[C] = \frac{2^b}{l} \cdot b \cdot \frac{1}{\frac{k-1}{k}} \Rightarrow b \cdot \frac{k}{k-1} < \frac{2^b}{l} \cdot b \cdot \frac{k}{k-1} < 2b \cdot \frac{k}{k-1}$$

Střední hodnota počtu hodů kostkou tak bude podobná jako je tomu u hodů mincí. Pro sudé *k* je počet hodů kostkou $< 2b$ a pro liché $< 2b \cdot \frac{k}{k-1}$ jelikož musíme započítat pravděpodobnost, že se generování bitu povede a nepadne pro lichý počet střední hodnota kostky.

Asymptotická složitost v závislosti na počtu hodů k-stěnnou kostkou bude $O(log_2 l)$ a na počtu hodů mincí celkově $O(log_2 l \cdot log_2 k)$. To proto, jelikož pro obě kostky musíme

vygenerovat počet bitů, odpovídající mocnině dvojky z čísla *k* a *l* případně o jeden vyšší. Vzhledem k tomu, že pomocí k-stěnné kostky simulujeme l-stěnnou, celkový počet hodů mincí se násobí.

Úkol 2:

Obecná mince:

```
T RandomProbability(T min, T max) {
   random_device rd;
   mt19937 gen(rd());
   uniform_real_distribution<> dis(min, max);
   return dis(gen);
}

int GeneralCoinFlip(double p_1) {
   auto flip = RandomProbability<double>(0.0, 1.0);
   if (flip <= p_1) {
       return 1;
   }
   return 0;
}
```

Jak bylo popsáno v prvním úkolu, tak obdobně i pro obecnou minci vygenerujeme číslo ležící v intervalu $[0, 1)$ a poté rozhodneme, zda překročila hodnotu parametru nebo ne.

Abychom úspěšně simulovali poctivou minci, musíme si představit, co z obecné vyplývá. Hned první vlastnost, kterou vypozorujeme je, že generuje bity 0 a 1 v pravděpodobnostech závislých na sobě. Přesněji pravděpodobnost pro bit 1 je *p*, z toho plyne, že pravděpodobnost bitu 0 bude přesně $1 - p$. Abychom získali poctivost pro obě hodnoty, budeme je muset vzájemně vyrušit. K vzájemnému vyrušení pravděpodobností nastane v jediný moment, a to když se společně “setkají”. Respektive ve chvíli, kdy padne jak 0, tak i 1.

Zde je algoritmus, který toho dosáhne:

```
int PerfectCoinFlipSimulation() {
   pair flips {GeneralCoinFlip(0.3), GeneralCoinFlip(0.3)};
   if (flips.first != flips.second) return flips.first;
   else return PerfectCoinFlipSimulation();
}
```

Pro ukázku jsou parametry pravděpodobnosti obecné mince nastaveny ručně. Ve skutečnosti pravděpodobnosti tak známe, ale můžeme přimhouřit oči a dělat, že je nevidíme 🙈 (nic to na principu chování algoritmu nezmění).

Algoritmus si dvakrát hodí obecnou mincí a v případě kdy padne 0 i 1, vrátí hodnotu prvního hodu v daném kole. V ostatních případech se rekurzivně opakuje, dokud se tak nestane.

Pravděpodobnost, že padnou opačné hodnoty (0 i 1) je součin pravděpodobností těchto nezávislých elementárních jevů v určitém kole. Musíme si ale uvědomit, že počet platných kombinací je roven dvěma ({0,1}{1,0}). Mnějme tedy celkovou pravděpodobnost P pro kterou platí $P = p \cdot (1 - p) + p \cdot (1 - p)$. To jsou naše platné kombinace. Jako příklad si vezměme ukázkové hodnoty z našeho algoritmu. Pravděpodobnost toho, že nastane alespoň jedna množina jevů pokud je $P(1) = 0,3$:
$P = 0,3(1 - 0,3) + 0,3(1 - 0,3) = 0,42$. Ovšem, zaručili jsme tímto postupem, že bude simulace vůči oběma hodnotám bitů 0 i 1, férová? Ano, a dokážeme to následovně. Označme si množinu všech platných kombinací B, a množinu konkrétní kombinace a tedy i výsledného bitu A. Dle definice podmíněné pravděpodobnosti z kuchařky jsme pak schopni dokázat, jaká je pravděpodobnost konkrétního bitu i kombinace, pokud nastal jev B, respektive všechny možné platné kombinace. Těch jsme schopni vzhledem k vysoké střední hodnotě dosáhnout. Ukažme si to i na našem konkrétním příkladu:

$P(A|B) = \frac{P(A \cap B)}{P(B)}; P(\{0,1\}|\{\{1,0\},\{0,1\}\}) = \frac{p \cdot (1-p)}{p \cdot (1-p) + p \cdot (1-p)} = \frac{0,3 \cdot (1-0,3)}{0,6 \cdot (1-0,3)} = \frac{1}{2} \Leftrightarrow 0,21$
Platí to i pro $P(\{1,0\}|\{\{1,0\},\{0,1\}\})$. Férovost je tedy zaručena!

Střední hodnota na hod jedné falešné mince bude $E[X] = \frac{1}{p} = \frac{1}{p \cdot (1-p)} = \frac{1}{0,21} \approx 4,76$.
Pro získání jedné úspěšné kombinace musíme hodit v průměru 4,76-krát falešnou mincí. V nejhorším případě budeme házet mincí znovu a znovu až do nekonečna. Pravděpodobnost této situace se ovšem vzhledem ke střední hodnotě blíží nule.

Úkol 3:

Simulovat obecnou minci poctivou také zvládneme, ovšem už hůře. Jak si zjednodušeně tuto situaci představit? Dostaneme nějakou pravděpodobnost *p* na vstupu pro bit 1. Pokud bude nižší než je pravděpodobnost, kterou pro bit 1 poskytuje férová mince, počet úspěšných hodů právě této hodnoty se musí zvýšit. Pro dosažení přesné pravděpodobnosti po mnoha hodech, rozhodneme o tom, zda se vrátí 1 nebo 0 podle toho jakou pravděpodobnost kombinace bitů 1 a 0 z poctivé mince jsme získali.

Pokud například nastanou dva stejné elementární jevy za sebou, pravděpodobnost této množiny je jejich součin. Výsledná kombinace hodů poctivé mince pak rozhoduje o tom, zda jsme hranici naší pravděpodobnosti ze vstupu dosáhli/překročili nebo ne. Nezávisle a podobně tak jako je tomu v kuchařce (zesilování pravděpodobnosti) budeme generovat sekvenci bitů a podle aktuálního bitu budeme posouvat naší hranici jaké výsledné pravděpodobnosti můžeme dosáhnout.

Vytvoříme si rozsah, který bude určovat horní hodnotu pravděpodobnosti, spodní a aktuální, která je mezi. Horní a spodní hodnoty budou značit okraje intervalu (na začátku bude horní hodnota 1, spodní 0 a aktuální výchozí bod 0,5), kam jsme pomocí budoucích bitů schopni se až dostat (do jaké pravděpodobnosti ji stlačit). Aktuální, jak už název napovídá, značí na jaké hodnotě pravděpodobnosti se aktuálně nacházíme. Abychom zajistili přesného dosažení pravděpodobnosti získaného na vstupu, musí se jeden z okrajů spolu s aktuální

hodnotou nacházet pod nebo nad požadovaným *p*. Konkrétně pro bit 1, se horní okraj pravděpodobnosti spolu s aktuálním (prakticky vzato celý rozsah i se spodní) musí nacházet pod hodnotou *p* abychom splnili zadanou pravděpodobnost a mohli vrátit 1. Pokud bychom posuzovali jen podle aktuální hodnoty, nedosáhli by jsme přesné pravděpodobnosti, jelikož by rozsah byl ještě velý a následující kombinace bitů by to mohla ještě zvrátit. Toho bychom ale nedocílili jelikož by algoritmus skončil.

Postup algoritmu bude tedy takový, že si vytvoří tento interval třemi proměnnými **lower_streak** = 0, **upper_streak** = 1, **streak** = 0,5. A poté bude házet mincí a podle toho jestli padlo 1 nebo 0 binárně zkracovat interval. Tento proces bude opakovat, dokud některým z okrajů nepřesáhne *p*.

Pokud v i-tém kroku padne 0, nastaví spodní hranici na aktuální, čímž binárně zmenší interval na polovinu a aktuální hodnotu aktualizuje na prostřední hodnotu tohoto nového intervalu.

Pokud padne 1, udělá to samé, akorát s tím rozdílem, že hodnotu aktuální hranice přiřadí horní hranici a opět aktualizuje aktuální jakožto střed nového intervalu. Jednoduché podmínky pak zjistí, zda se celý interval nachází pod nebo nad určitou pravděpodobností *p* a pokud ano, vrátí hodnotu bitu a algoritmus skončil.

Simulace obecné mince poctivou:

```
int GeneralCoinFlipSimulation(double p) {
   double lower_streak = 0.0;
   double upper_streak = 1.0;
   double streak = (lower_streak + upper_streak) / 2;

   while (true) {
       int flip = PerfectCoinFlip();
           if (flip == 0) {
               lower_streak = streak;
               streak = (upper_streak + streak) / 2;
               if (lower_streak > p && streak > p) return flip;
           }
           else {
               upper_streak = streak;
               streak = (lower_streak + streak) / 2;
               if (streak <= p && upper_streak <= p) return flip;
           }
       }
}
```

Střední hodnota počtu hodů mincí se vzhledem k neustále pulícímu se intervalu blíží 2, tedy $x = 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} + 3 \cdot \frac{1}{8} + 4 \cdot \frac{1}{16} + ... = 2$

Asymptotická složitost je $O(1)$, jelikož není ovlivněna vstupem a aritmetické operace jsou konstantní.

Důkazem správnosti je výsledná kombinace bitů udávající pravděpodobnost jejich sekvence a transformace intervalu podle ní. Nemůže nastat situace, kdy bychom skončili dřív, než je zaručené, že při dalších hodech už bychom se nemohli dostat nad nebo pod zadanou pravděpodobnost *p*. To je nemožné, protože po každém hodu je interval jednoznačně určený a nelze zpětně měnit sekvenci padlých bitů ani posunutí intervalu.

Úkol 4:

Ten bohužel jsem nestihl.